

Group 14:
CB.AI.U4AID25104-A SHASHANK ROYAL
CB.AI.U4AID25125-K ARAVIND GUPTHA
CB.AI.U4AID25126-M DINESH KUMAR
CB.AI.U4AID25129 -M SUJAN

Table of contents :

- Introduction
- Problem statement and Proposed Solution
- Reasearch Angle
- objectives and methodologies
- work to be done
- work division

Introduction:-

SmartBasket Neo is an AI-powered e-commerce recommendation platform that recreates a real-world shopping experience like Amazon or Flipkart. It uses machine learning to deliver real-time personalized product suggestions based on each user's browsing and purchasing behavior. With intelligent algorithms and a user-friendly interface, SmartRecomendation enhances engagement, improves customer satisfaction, and enables smarter shopping.

problem statement & proposed solution:-

The core motivation comes from 3 real-world problems in e-commerce:

Generic product listings shown to everyone	**→ Personalized ML model per user trained on their own behavior**
Users can't find what they want via keyword search	**→ Semantic AI search using sentence embeddings (FAISS)**
No way to measure if recommendations actually help	**→ A/B Testing — Group A gets ML recommendations, Group B gets trending products**

In short: SmartBasket Neo is a working proof-of-concept that demonstrates how modern e-commerce platforms use ML to move from "show everyone the same products" to "show each person what they personally will buy."

Research Angle — Federated Learning

Per-user local models are actually the future direction called Federated Learning — used by

- Google (Gboard keyboard predictions)
- Apple (Siri personalization)
- Healthcare (private patient models)

our project simulates this concept — each user's model trains only on their own data, which also means better privacy.

"while industry systems like Amazon use a single global model for scalability, our research explores a hybrid approach — combining a personalized local CatBoost model trained on individual user behavior and global Catboost model and with a global Neural Collaborative Filtering model. This mirrors emerging Federated Learning architectures where user privacy and personalization are prioritized"

Objectives
AND METHODOLOGY

Objectives

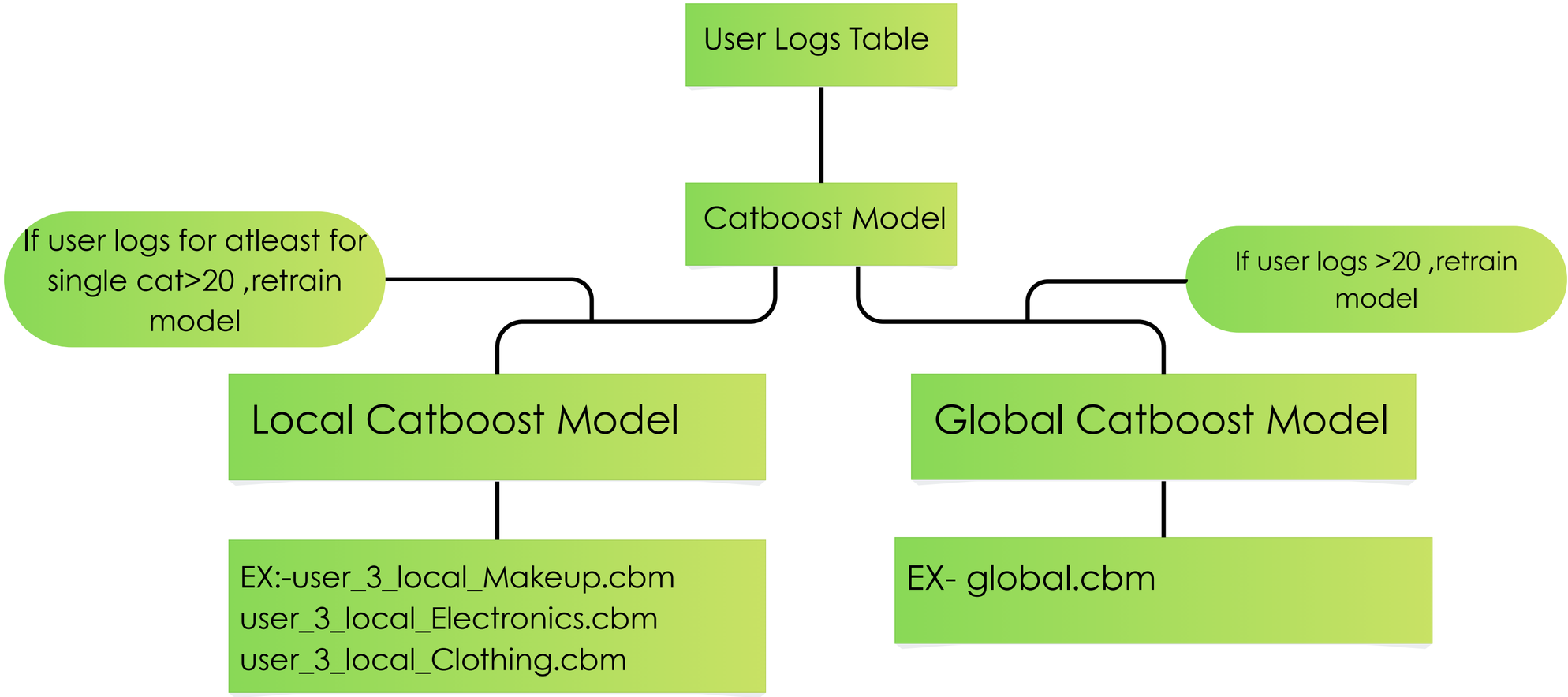
1. Build a Personalized Product Recommendation System

2.Implement Neural Collaborative Filtering (NCF)

3. Enable Semantic / Meaning-Based Search

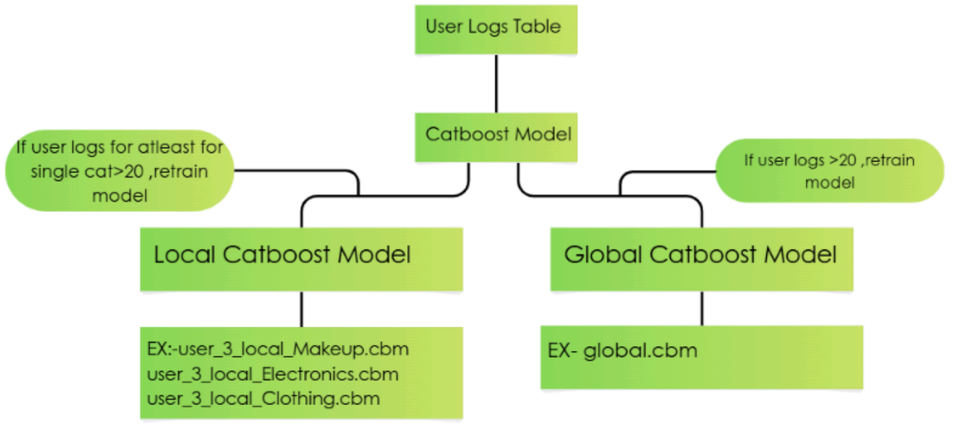
4.Deploy a Full-Stack Web Application

DESIGNED
ARCHITECTURE
to build a Personalized Product Recommendation System


User Logs Table
Catboost Model
If user logs for atleast for single cat>20 ,retrain model
If user logs >20 ,retrain model
Local Catboost Model
Global Catboost Model
EX:-user_3_local_Makeup.cbm
user_3_local_Electronics.cbm
user_3_local_Clothing.cbm
EX- global.cbm

Methodology:-

- Tier A (Local CatBoost): A supervised gradient-boosting model trained exclusively on an individual user's data. It exists in two forms:
  - Global CBM: Trained on all of the user's logs to learn general preferences.
  - Local CBM: Trained only on logs within a specific category (e.g., Makeup) for ultra-precise, domain-specific ranking.



why catboost only?

E-commerce data is highly tabular and categorical (brands, categories, strings). Deep learning struggles with small tabular datasets. CatBoost is specifically designed for categorical data without requiring manual one-hot encoding, and it handles missing values (NaNs) natively. It is incredibly fast at both training and inference.

Why Full Retrain Every Time?

Preventing "Catastrophic Forgetting"

If you only train a model on the newest 20 logs, the model physically overwrites its internal weights to fit those 20 logs. It will completely forget the behavioral patterns it learned from the first 100 logs.

- Example: A user buys a laptop in month 1, and buys a mouse in month 2. If you only train incrementally on month 2, the model forgets the user owns that specific laptop. A full retrain ensures the model sees the entire journey and context.

Simplicity and Stability

Incremental learning (online learning) requires highly complex architecture to manage learning rates decay, concept drift, and catastrophic forgetting mitigation. Batch learning (full retrain) is mathematically stable, highly reproducible, and much easier to debug for a proof-of-concept project.

The Data Scale Permits It

In massive systems like Amazon (billions of logs), full retraining is too computationally expensive, so they are forced to use complex incremental learning streams. However, in our system, the data size per user is small (a few hundred to a few thousand logs max). Training a CatBoost model on 1,000 rows takes less than 2 seconds. Therefore, full retraining is perfectly viable and guarantees the highest possible accuracy

Data preprocessing

How Data Preprocessing happens inside Catboost

Before we hand the data to CatBoost, we structure it so the model understands what we want it to learn.

1. Generating the Target (Labels)
2. Handling the Extreme Imbalance
3. Categorizing the Columns

What CatBoost does INTERNALLY

1. Handling Missing Data (NaNs)
2. Categorical Encoding
3. Text Feature Processing
4. Handling Categorical Combinations

1. Create the Target

- Convert user actions into a binary label:
  - Purchase / Cart → 1 = Positive intent
  - View / Search → 0 = Negative/Neutral intent

2. Handle Class Imbalance

- E-commerce usually has many more 0s than 1s.
- Calculate scale_pos_weight to give more importance to the rare positive class.
- This prevents the model from simply predicting 0 for everyone.

3. Define Feature Types

Tell CatBoost what each column represents:

- Categorical: Brand, Category, EAV features
- Text: Product Title
- Numerical: Price, Rating, Reviews

1. Missing Values

- EAV features can contain NaN.
- With nan_mode="Min", CatBoost treats missing values as a separate pattern.
- The model can therefore learn: "This feature is missing → this may indicate something."

2. Categorical Features

- CatBoost does not require traditional One-Hot Encoding.
- It uses Ordered Target Statistics to learn how categories relate to the target.
- Example:
  - Apple → higher purchase tendency
  - Sony → lower purchase tendency
- It calculates these statistics in an ordered way to reduce target leakage.

3. Text Features

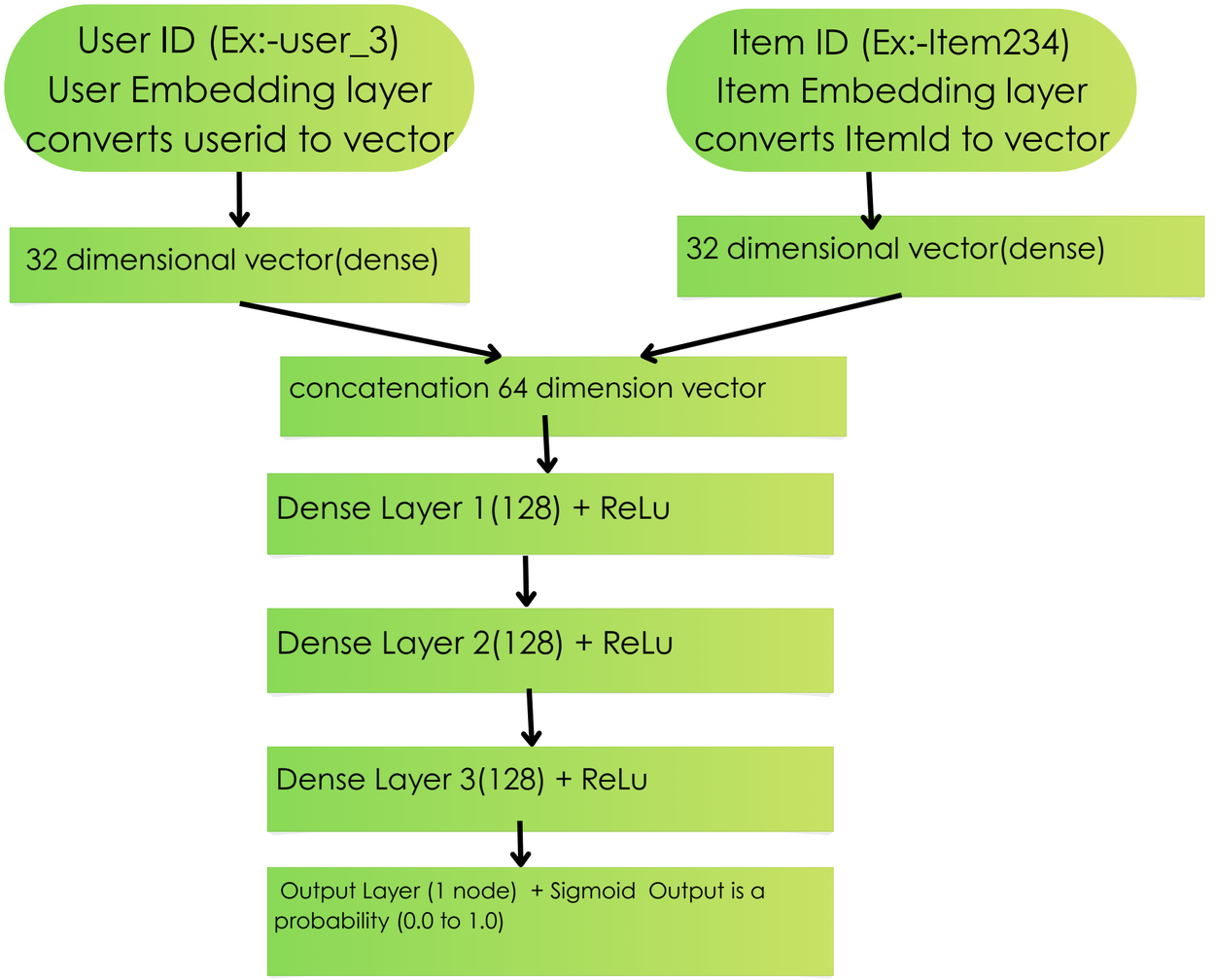
- Product titles are processed as text rather than a single category.
- CatBoost extracts useful information from words/tokens.
- Example:
  - "Gaming Laptop RTX 4060"
  - The model can learn that words like "Gaming" or "RTX" may be associated with certain user behavior.

4. Feature Combinations

- CatBoost can automatically discover interactions between categorical features.
- Example:
  - Brand = Apple
  - Category = Headphones
  - Apple + Headphones may have a stronger purchase tendency than either feature alone.

DESIGNED
ARCHITECTURE
Implement Neural Collaborative Filtering (NCF)

Neural Collaborative Filtering is a deep learning architecture that replaces the traditional "dot product" of Matrix Factorization with a neural network.



Methodology

NCF is "Collaborative": NCF learns from the entire community. If NCF notices that User A and User B both buy similar laptops, and User A suddenly buys a camera, NCF learns that User B might also want that camera.

Why Neural over Traditional Matrix Factorization? Traditional methods just multiply the User Vector by the Item Vector. That is a linear operation. E-commerce behavior is highly non-linear. The deep layers (MLP) in NCF can learn complex, hidden interactions that simple math misses.

Complications & Simplifications:-

1.The Extreme Sparsity Problem:- We have 1.4 million products . A highly active user might view 100 products that means 99.99% of the user item matrix is completely empty .Neural networks struggle to learn when almost all data is missing .
We do not train NCF every time a user clicks something. NCF is trained globally offline as a heavy background job.
2.The Cold Start Problem:- If a brand new user registers today, they have no ID embedding in the trained network. The NCF model literally cannot make a prediction for them until it is fully retrained.
Fallback Mechanisms (Handling Cold Start): If NCF encounters a completely new user, the system gracefully falls back to showing "Global Trending/Best Sellers" until the next global retraining cycle happens.
3.Computational Expense: Unlike CatBoost, which trains on one user's data in 2 seconds, training a global neural network on millions of logs across all users takes massive computational power (GPUs) and time (minutes to hours). It cannot be done in real-time.
Memory-Efficient Embeddings: Instead of one-hot encoding 1.4 million products (which would crash the RAM), we use PyTorchnn.Embedding layers to map sparse IDs directly into dense 32-dimensional vectors, keeping memory usage drastically low.

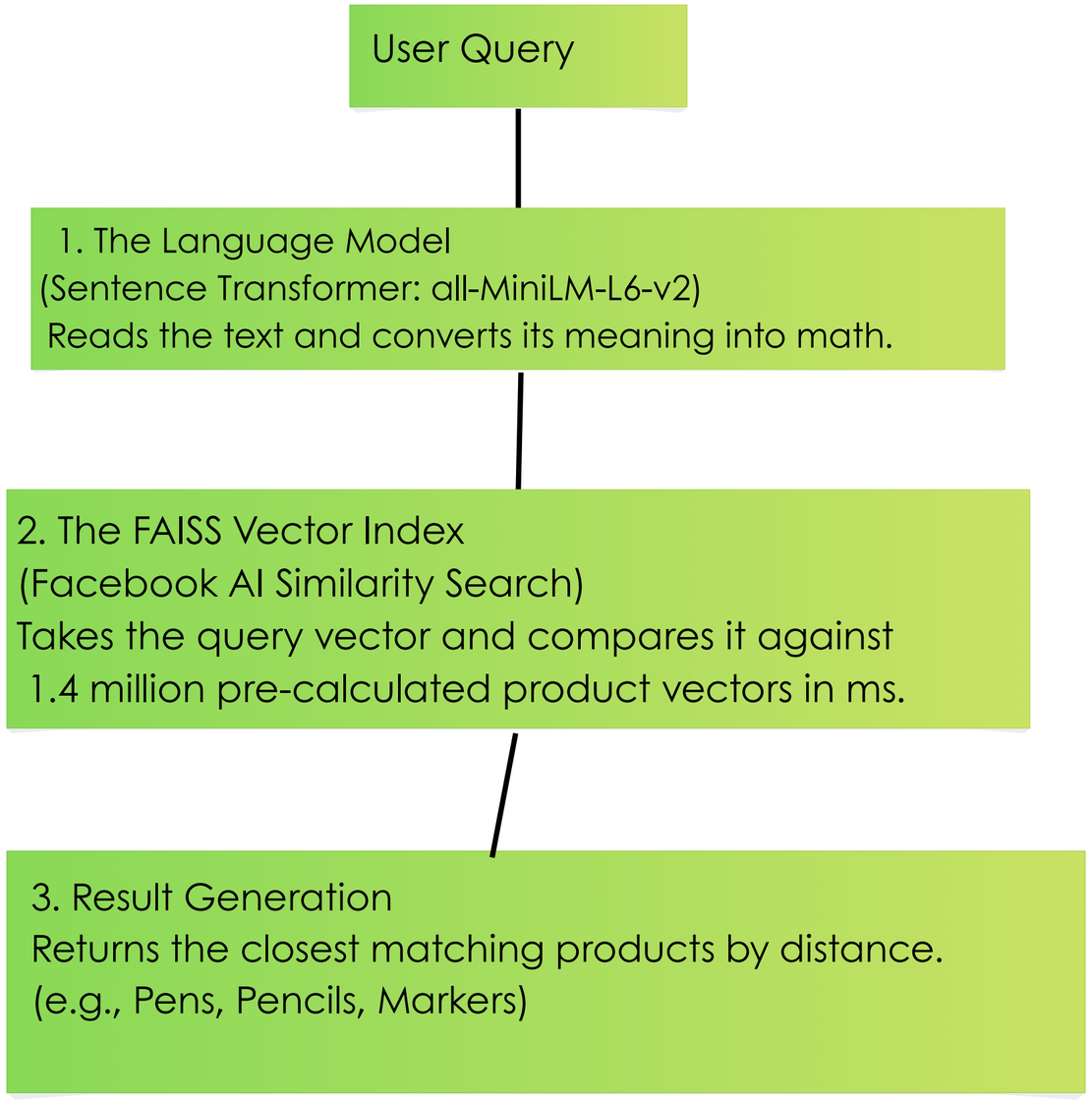
Data Processing for NCF

How do we train a neural network on e-commerce logs?

Implicit Feedback: We don't have 5-star ratings for everything. We only have logs (views/carts).
If a user interacted with an item, we label it as 1 (Positive Sample).
Negative Sampling (Crucial Step): If a user didn't click an item, does it mean they hate it? No, maybe they just didn't see it. However, the neural network needs negative examples to learn what not to recommend.
Processing Trick: For every 1 (interaction), the system randomly picks 4 products the user has never seen and labels them 0 (Negative Sample).
Loss Function: The network is trained using Binary Cross Entropy (BCE) Loss, which calculates the difference between the predicted probability and the actual label (1 or 0), adjusting the weights via backpropagation.

PROPOSED
ARCHITECTURE

Enable Semantic / Meaning-Based Search


User Query
1. The Language Model
(Sentence Transformer: all-MiniLM-L6-v2)
Reads the text and converts its meaning into math.
2. The FAISS Vector Index
(Facebook AI Similarity Search)
Takes the query vector and compares it against
1.4 million pre-calculated product vectors in ms.
3. Result Generation
Returns the closest matching products by distance.
(e.g., Pens, Pencils, Markers)

2. Methodology: Why Only This Method?

If standard SQL database search (shoes) works, why go through the trouble of building a complex semantic search?

The Problem with Standard Search (Keyword Matching): If a user searches for "running shoes for flat feet", standard SQL looks for those exact words. If a product is titled "Orthopedic Athletic Footwear", standard search will return zero results, even though it is exactly what the user wants. Keywords match letters, not intent.

Why Semantic Search? Semantic search maps concepts into a 3D-like space (actually 3-dimensional space).

In this space, the math vector for "Running Shoes" is physically placed right next to the math vector for "Athletic Footwear".

When the user searches, the system just looks at that physical spot in the vector space and grabs everything nearby.

Conclusion: We chose this method because human language is messy. People use synonyms, slang, and descriptions instead of exact product names. Semantic search is the only way to understand what the user means, not just what they typed.

3. Data Processing (How it actually works)

You cannot run a deep learning model on 1.4 million products every time a user hits "Search". It would take hours. Here is how the data processing is split:

Step A: Offline Processing (Done once in the background)

1. A background script loops through all 1.4 million products in the database.
2. It passes every single product_title into the Sentence Transformer model.
3. The model outputs a 3-dimensional array of numbers (the embedding) for each product.
4. All 1.4 million arrays are packed together and saved into a special highly-optimized file called a FAISS Index (faiss_index.bin).

Step B: Real-Time Processing (When the user searches)

1. The user types "something for ears".
2. The API quickly passes just that one short sentence into the Transformer model, getting a single 3-dimensional query vector.
3. FAISS instantly calculates the geometric distance between the query vector and the 1.4 million product vectors in memory, returning the Top 20 closest matches in under 50 milliseconds.

work to be done ?

We have to design architecture of full stack web application .
There is a limitation while sampling a data we have to work on it
We have to still go in detail about sementic search
testing and training of our ML models

On going decisions:-

we have an idea to use sementic search while sampling the data which we are yet to take decision

work division

The Catboost model implementation & validation.
Integrating With web application.
Dealing with only local catboost models and testing and back end of Web application and related data preprocessing

The Catboost model implementation & validation.
Integrating With web application.
Dealing with only global catboost models and Front end of web application and related data preprocessing

The NCF implementation
Enable semantic based search testing
Integrating with web application Dealing with Front end part of web application and related data preprocessing

NCF implementation and testing
Enable Semantic-Based Search.
Implemention and Dealing with back end part of web application and related data preprocessing